SEMESTER PROJECT PROPOSAL

Speaker-Attributed Meeting Summarization & Action Item Extraction System

NLP / Voice AI — 1-Month Project Plan

Domain: Natural Language Processing · Automatic Speech Recognition · Multi-Speaker Understanding
Scope: Minimum Viable Version (Core deliverable) + Future Extensions (documented separately)
Data: AMI Meeting Corpus + ICSI Meeting Corpus — fully public, annotated, no data collection required
Duration: 1 months

1. EXECUTIVE SUMMARY

Most meeting summarization tools treat transcripts as a flat bag of sentences — losing the crucial context of who said what, who committed to what, and who disagreed with whom. This project builds a system that preserves speaker identity throughout the entire NLP pipeline, from raw audio to structured report.

The core deliverable (Minimum Viable Version) is a fully evaluable, end-to-end pipeline that takes a multi-speaker meeting recording and produces a speaker-attributed structured summary with extracted action items and decisions — all benchmarked against ground-truth annotations from the AMI corpus.

2. PROBLEM STATEMENT

Existing meeting summarization approaches suffer from three key limitations:

- Attribution loss — summaries say 'the team decided X' instead of 'Sarah proposed X and John opposed it'
- No commitment tracking — action items are frequently missed or not linked to the responsible speaker
- No disagreement modeling — conflicting positions between participants are flattened into consensus statements

These gaps mean summarization tools are useful for recall but not for accountability. This project directly addresses all three by making speaker identity a first-class citizen throughout the pipeline.

3. PROJECT OBJECTIVES

Primary Objectives (Minimum Viable Version)

- Build a diarization + ASR pipeline that produces speaker-attributed transcripts with measurable accuracy (DER < 15%, WER < 20% on AMI test set)
- Develop and fine-tune an action item detection classifier using AMI's ground-truth annotations, targeting F1 > 0.70
- Develop and fine-tune a decision detection classifier on AMI annotations, targeting F1 > 0.65
- Build a disagreement detection module using NLI-based cross-speaker contradiction scoring
- Generate structured, speaker-attributed meeting summaries using an LLM with a grounded prompting strategy
- Evaluate all components quantitatively and write a complete project report

Secondary Objectives (if time permits)

- Add a TTS layer for spoken summary output
- Build a simple Q&A interface for querying the meeting transcript

4. SCOPE — MINIMUM VIABLE VERSION

The following defines precisely what will and will not be built in the core project deliverable.

4.1 In Scope

Component	Description
Speaker Diarization	Who spoke when — using pyannote.audio pretrained model on AMI IHM recordings
Automatic Speech Recognition	Transcription of each speaker segment using OpenAI Whisper (medium model)

Action Item Extraction	Classification + structured extraction: {assignee, task, deadline} using fine-tuned transformer on AMI labels
Decision Detection	Binary classifier to identify decision statements, fine-tuned on AMI decision annotations
Disagreement Detection	NLI-based cross-speaker contradiction scoring using DeBERTa-v3 within a sliding context window
Summary Generation	LLM-based structured summary with speaker attribution, grounded in extracted structured data
Evaluation	Full quantitative eval: DER, WER, F1 (action items), F1 (decisions), ROUGE-L and BERTScore (summaries)

4.2 Explicitly Out of Scope (MVP)

The following are intentionally excluded from the Minimum Viable Version to ensure deliverability within the semester timeline.

- Text-to-Speech (TTS) voice output — spoken summary generation
- Real-time / live audio processing — all processing is on pre-recorded files
- Conversational Q&A interface — asking follow-up questions about the meeting
- Paralinguistic / acoustic stress analysis — pitch, tempo, jitter features
- Graph database (Neo4j) — in-memory networkx is sufficient for MVP
- Full PEACE interview model — not applicable to meeting domain
- Mobile / web deployment — local execution only

These are documented as future extensions in Section 8.

5. SYSTEM ARCHITECTURE

The system follows a sequential modular pipeline where each component can be evaluated and replaced independently.

Pipeline Overview

```
Audio File (.wav / .mp4)
        ↓
[1] Speaker Diarization         pyannote.audio 3.1 — who spoke when
        ↓
```

```
[2] ASR Per Segment              Whisper medium — what was said
       ↓
[3] Speaker-Attributed Transcript
       ↓
┌──────────────────────────────────────┐
│  NLP Extraction Layer                │
│  [4a] Action Item Detector           │
│  [4b] Decision Detector              │
│  [4c] Disagreement Detector (NLI)    │
└──────────────────────────────────────┘
       ↓
[5] Structured Summary Generator     LLM (GPT-3.5 / Claude Haiku)
       ↓
[6] Structured Report Output         JSON + Markdown / PDF
```

5.1 Component Details

Component	Technical Details
Diarization	pyannote/speaker-diarization-3.1 via HuggingFace. Input: IHM audio. Output: (start, end, speaker_id) segments. Evaluation: Diarization Error Rate (DER).
ASR	OpenAI Whisper medium model. Applied per-segment after diarization. Output: text per speaker turn. Evaluation: Word Error Rate (WER) vs AMI gold transcripts.
Action Item Classifier	Fine-tune bert-base-uncased or roberta-base on AMI action item labels. Features: future tense, commitment verbs, assignee NER. Output: binary + structured extraction.
Decision Classifier	Same fine-tuning approach on AMI decision labels. Signal words: 'agreed', 'decided', 'we will go with'. Output: binary classification per utterance.
Disagreement Detector	cross-encoder/nli-deberta-v3-base. Sliding context window (k=5 turns). Compare each new utterance vs prior turns from different speakers. Threshold: contradiction score > 0.65.
Summary Generator	LLM call (GPT-3.5-turbo or Claude Haiku for cost control). Structured prompt with extracted data. Output: 5-section formatted summary with speaker attribution throughout.

6. DATASETS & EVALUATION

6.1 Datasets

Dataset	Details
AMI Meeting Corpus	100+ hours of scenario-based meetings. Individual Headset Mic (IHM) recordings for clean audio. Ground truth: transcripts, speaker turns, action item labels, decision labels, abstractive summaries. Primary dataset.
ICSI Meeting Corpus	75 hours of real academic meetings. More naturalistic than AMI. Use as out-of-domain test set to evaluate generalization.

6.2 Evaluation Plan

Component	Metric	Baseline Target	Dataset
Diarization	DER	< 15%	AMI test set
Transcription	WER	< 20%	AMI transcripts
Action Item Detection	F1	> 0.70	AMI action labels
Decision Detection	F1	> 0.65	AMI decision labels
Summary Quality	ROUGE-L / BERTScore	ROUGE-L > 0.35	AMI abstractive summaries
Disagreement Detection	Precision / Recall	Precision > 0.60	Manually created test cases (30)
End-to-End	Qualitative demo	Coherent, attributed output	5 held-out AMI meetings

Note on disagreement evaluation: AMI does not have pre-labeled disagreement annotations. A small test set of ~30 meeting excerpts with manually planted disagreements will be constructed — this is approximately 4-6 hours of work, not the multi-week data problem that affects other project ideas.

7. PROJECT TIMELINE

Week 1 — Foundation

- Environment setup: install pyannote, Whisper, HuggingFace transformers, spaCy, networkx
- Download and preprocess AMI IHM recordings and annotations
- Build diarization + ASR pipeline, measure DER and WER on AMI test set
- Build speaker-attributed transcript merger (collapse consecutive same-speaker turns)
- Deliverable: working audio-to-attributed-transcript pipeline with baseline metrics

Week 2 — NLP Extraction Layer

- Fine-tune action item classifier on AMI labels; evaluate F1 on held-out split
- Fine-tune decision classifier on AMI labels; evaluate F1
- Build structured action item extractor: {assignee, task, deadline} using NER + LLM
- Build disagreement detection module using DeBERTa NLI; create small evaluation set
- Deliverable: all three extractors running end-to-end on attributed transcripts

Week 3 — Summary Generation & Integration

- Design and iterate on LLM prompt for structured, speaker-attributed summary generation
- Integrate all pipeline components into a single runnable script
- Run full evaluation on AMI test set across all metrics
- Build a 5-meeting end-to-end demo with qualitative analysis
- Deliverable: complete working system with full quantitative evaluation

Week 4 — Polish, Writing & Submission

- Error analysis: examine failure cases for each component
- Ablation study: measure impact of disagreement detection on summary quality
- Write project report / paper (target: 8-10 pages, ACL format)
- Record demo video showing the system processing a real meeting
- Deliverable: final report, demo, and codebase

Buffer Strategy

If Month 2 runs over, use AMI's ground-truth diarization labels as input to the NLP layer (instead of the predicted diarization). This decouples the components and keeps the project moving. Resume integrating real diarization output in Month 3.

8. FUTURE EXTENSIONS (Excluded from MVP)

The following components were deliberately excluded from the Minimum Viable Version to protect the timeline. They are documented here as natural extensions that could be pursued after submission or in a follow-up project.

8.1 Text-to-Speech Voice Output

Add a TTS layer (Coqui TTS or ElevenLabs API) to read the structured summary aloud after the meeting. Could use different voice profiles per section (overview, action items, decisions). This makes the system a true 'voice agent' rather than just a text pipeline. Estimated additional effort: 2-3 weeks.

8.2 Conversational Q&A on the Meeting

Build a RAG layer on top of the speaker-attributed transcript. Users can ask natural language questions like 'What did John say about the budget?' or 'Were there any disagreements about the launch date?' and get grounded, speaker-attributed answers. Stack: simple embedding-based retrieval (FAISS) + LLM. Estimated additional effort: 3-4 weeks.

8.3 Real-Time / Live Audio Processing

Replace the offline batch processing with a streaming pipeline using pyannote's online diarization and Whisper's streaming mode. The main challenge is latency — maintaining sub-3-second lag while keeping diarization accurate. Requires a fundamentally different architecture (chunked audio buffers, rolling context). Estimated additional effort: 6-8 weeks.

8.4 Paralinguistic Feature Layer

Extract acoustic features (pitch variance, speech rate, pause duration, vocal energy) per speaker turn using openSMILE or pyAudioAnalysis. Use these as soft signals to flag potential high-tension exchanges or disengagement. These features do not serve as lie detection — they are provided as reviewer annotations only. Estimated additional effort: 3-4 weeks.

8.5 Graph-Based Knowledge Representation

Replace the in-memory data structures with a proper knowledge graph (Neo4j or RDFLib) where meeting entities, commitments, and decisions are represented as nodes and edges. This enables richer queries ('find all commitments John has made across 5 meetings that are still unresolved') and supports multi-meeting longitudinal analysis. Estimated additional effort: 4-6 weeks.

8.6 Multi-Meeting Longitudinal Tracker

Track action items and decisions across multiple meetings over time. Flag when a previous action item was never completed, when the same disagreement recurs, or when decisions contradict each other across sessions. Requires persistent storage and entity resolution across meetings. This is the most impactful extension and most suitable for a follow-up research paper.

9. TECHNICAL STACK

Category	Tools / Libraries
ASR	openai-whisper (medium model)
Diarization	pyannote.audio 3.1 (HuggingFace)
NLP / Transformers	HuggingFace Transformers, bert-base-uncased, roberta-base, cross-encoder/nli-deberta-v3-base
NER & Text Processing	spaCy (en_core_web_trf)
Data Structures	networkx (in-memory claim graph), pandas
LLM API	OpenAI GPT-3.5-turbo (development) / GPT-4 or Claude API (final eval)
Audio Processing	librosa, pydub, soundfile
Evaluation	jiwer (WER), pyannote.metrics (DER), rouge-score, bert-score
Development	Python 3.11, PyTorch, Jupyter Notebooks, Git
Dataset Access	HuggingFace Datasets (edinburghcstr/ami)

10. RISK ASSESSMENT & MITIGATIONS

Risk	Likelihood	Impact	Mitigation
Diarization accuracy insufficient on overlapping speech	Medium	High	Use AMI IHM (individual mics) instead of array mics. Fall back to ground-truth diarization for NLP evaluation if needed.
LLM API costs exceed budget during iteration	Medium	Low	Use Claude Haiku or GPT-3.5 during development. Only call GPT-4 for final evaluation run (~50 calls).
Fine-tuning underperforms	Low	Medium	Use a larger pretrained model (RoBERTa-large). Add data augmentation.

on small AMI split			Report baseline (zero-shot LLM) as comparison.
Disagreement evaluation too subjective	Medium	Medium	Define a strict annotation guide. Have two annotators label the 30 test cases and report inter-annotator agreement (Cohen's kappa).
Scope creep toward extensions	High	High	Strictly treat Section 8 items as post-submission work. Lock scope after Month 2.

11. SUCCESS CRITERIA

The project will be considered successfully complete if ALL of the following are met:

✓ End-to-end pipeline runs on any AMI meeting file without manual intervention
✓ DER < 15% and WER < 20% on the AMI IHM test set
✓ Action item F1 > 0.70 on held-out AMI annotations
✓ Decision detection F1 > 0.65 on held-out AMI annotations
✓ Summary output is speaker-attributed (every decision and action item names a speaker)
✓ Disagreement detector achieves precision > 0.60 on the manually created test set
✓ Complete written report (8-10 pages) with methodology, results, and error analysis
✓ Working demo on 5 held-out AMI meetings

12. EXPECTED CONTRIBUTIONS & PORTFOLIO VALUE

This project makes the following distinct contributions beyond standard coursework:

- Speaker attribution as a first-class pipeline citizen — not a post-processing afterthought
- Combined fine-tuning + NLI approach to structured meeting understanding with rigorous AMI-based evaluation
- A disagreement detection module for meeting transcripts — underexplored in the literature relative to decision and action item detection
- An ablation study quantifying the contribution of each pipeline component to summary quality

From a portfolio standpoint, this project demonstrates: end-to-end audio NLP pipeline engineering, fine-tuning on structured annotation data, NLI-based reasoning, LLM prompt

engineering with grounding, and rigorous quantitative evaluation — all of which are directly relevant to NLP engineering roles at AI companies.

Target submission venues: ACL Student Research Workshop, EMNLP Industry Track, or Interspeech.

End of Proposal